

# Assignment 3: Evaluating Single-Agent and Multi-Agent AI Systems

## Objective

The objective of this assignment is to evaluate when an agent-based design is useful and when it adds unnecessary complexity. I compare two approaches for the same AI task: a single-agent social media generator and a multi-agent social media generator. Both systems receive the same user prompts and produce comparable outputs: topic, caption, media prompt, generated media path, QA result, browser/upload result, and execution logs.

The main research question is:

**Does a multi-agent architecture improve output quality and reliability enough to justify its extra coordination cost?**

## Part 1 - Experimental Setup

### Task Definition

The task is automated social media post generation. Given a user prompt, the system should:

- Understand the requested topic and platform.
- Decide whether research, writing, image generation, QA, and upload are needed.
- Generate a caption.
- Generate or prepare a visual prompt and media file when required.
- Check the output for quality and safety.
- Simulate publishing to a local demo upload page.

This task is appropriate for comparing agentic systems because the amount of work changes depending on the prompt. For example, a vague prompt like “Make a post about Pilates” requires planning, caption writing, media generation, QA, and upload. A detailed prompt like “Use this exact caption with a reformer bed image” should preserve the supplied caption and avoid unnecessary writing.

### System A: Single-Agent Baseline

The single-agent system is implemented in `single_agent_version/`. It uses one agent, `SingleSocialMediaAgent`, to handle all responsibilities:

- Planning
- Topic extraction
- Caption writing
- Media prompt construction
- Media generation
- QA checking
- One repair attempt
- Local demo upload

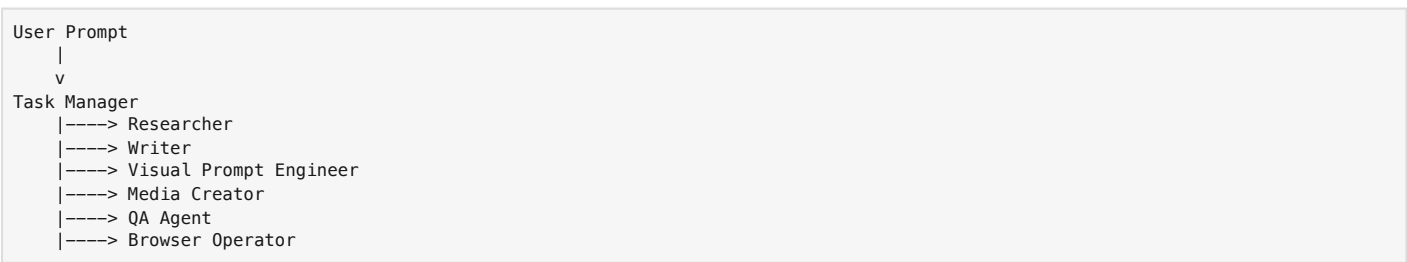
This is the simpler baseline. It has fewer moving parts, fewer logs, and fewer model-like steps. Its weakness is that one agent must keep all requirements in its own context, which can cause mixed instructions to be collapsed into one interpretation.

### System B: Full Multi-Agent System

The full multi-agent system uses specialized agents coordinated by a Task Manager:

| Agent                  | Responsibility                                                              |
|------------------------|-----------------------------------------------------------------------------|
| Task Manager           | Routes work, validates each step, skips unnecessary agents, triggers repair |
| Trend Researcher       | Produces trend keywords and hashtags when the prompt is vague               |
| Content Writer         | Writes captions and rough media prompts                                     |
| Visual Prompt Engineer | Converts rough image requests into professional photo prompts               |
| Media Creator          | Calls image APIs or local image generation                                  |
| QA Agent               | Checks caption/media quality and safety                                     |
| Browser Operator       | Simulates upload to the local demo page                                     |

The architecture follows a hub-and-spoke pattern:



The Task Manager dynamically skips agents. For example, if the user already provides an exact caption, the Writer is skipped. If the user requests text only, the Media Creator and Browser Operator are skipped.

## Comparable Inputs and Outputs

Both systems were evaluated on the same five test cases.

| ID | Input Type                      | Prompt Summary                                                                | Expected Output                                              |
|----|---------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------|
| T1 | Vague prompt                    | "Make a post about Pilates."                                                  | English Pilates caption, generated media, demo upload        |
| T2 | Detailed English product prompt | Instagram jewelry seller asks for close-up woman's ear with 3 silver earrings | English jewelry caption, silver earrings visual, demo upload |
| T3 | Turkish product prompt          | Outdoor sports equipment company asks for a man hiking in mountains           | Turkish caption, outdoor equipment visual, demo upload       |
| T4 | Exact caption + image           | Use exact text "Join my class!" with a reformer bed picture                   | Exact caption preserved, reformer/Pilates image, demo upload |
| T5 | Text-only draft                 | Write only a caption for a luxury jewelry launch, no image, draft only        | English caption, no media, no upload                         |

Expected comparable output fields:

- topic
- caption
- media\_prompt
- optimized\_media\_prompt
- media\_path
- qa\_status
- browser\_status
- execution logs

## Part 2 - Metrics Definition

I used automatic metrics that can be computed from each run. The goal is not only to judge final quality, but also to measure coordination overhead and reliability.

| Metric         | Why It Is Relevant                                                  | Measurement Method                                                                       |
|----------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Correctness    | The output must match the requested topic, language, and QA result. | Average of topic score, language score, and QA score.                                    |
| Completeness   | A usable post needs all required pieces.                            | Checks caption, required media, required upload, and QA status.                          |
| Consistency    | Caption, topic, and media prompt should describe the same request.  | Checks whether expected topic terms appear across topic/caption/media fields.            |
| Latency        | Practical systems must finish in acceptable time.                   | Wall-clock runtime in milliseconds.                                                      |
| Log Count      | More logs indicate more coordination overhead.                      | Number of execution log entries.                                                         |
| LLM Step Proxy | Direct token usage can vary, so model-like steps approximate cost.  | Counts Researcher, Writer, Visual Prompt Engineer, QA, or one single-agent content step. |
| Failure Rate   | Automation must handle errors reliably.                             | Percentage of runs with exception or non-approved QA status.                             |
| Overall Score  | Gives a compact comparison.                                         | Weighted score across correctness, completeness, consistency, media, and upload.         |

Important limitation: this experiment does not use a human visual-quality score as part of the primary metric. The media metric checks whether an image was produced and whether the text fields stay aligned with the requested topic. It does not prove that the generated image is aesthetically better. For that reason, image quality should be discussed as a qualitative observation, not as a fully measured result.

The overall score was computed as:

```
overall = 0.35 * correctness
          + 0.25 * completeness
          + 0.20 * consistency
          + 0.10 * media_score
          + 0.10 * upload_score
```

These weights prioritize semantic quality first, then whether the system produced all required artifacts, and finally media/upload success.

## Part 3 - Experiment Execution

The experiment runner is implemented in:

```
projects/Smart_Social_Media_Agent_Orchestrator/tools/run_assignment3_experiment.py
```

Primary deterministic experiment:

```
/Users/alki/.pyenv/versions/3.13.2/bin/python3 \
  projects/Smart_Social_Media_Agent_Orchestrator/tools/run_assignment3_experiment.py \
  --provider mock \
  --image-provider pillow
```

This produced 15 total executions:

- 5 test cases

- 3 systems per test case
- 1 repeat

The primary run used `provider=mock` and `image_provider=pillow`. This makes the experiment deterministic and repeatable. Real API services can introduce temporary failures, latency spikes, and model variability, so I used the mock/Pillow run as the main scientific baseline.

Primary output directory:

```
projects/Smart_Social_Media_Agent_Orchestrator/outputs/assignment3_experiment/20260514_150714
```

Generated artifacts:

- `results.json`
- `results.csv`
- `results_table.md`
- `aggregate_summary.json`

I also ran a supplementary real API trial with NVIDIA/Mistral providers. That run is discussed separately because external API availability affected some results.

## Part 4 - Results and Analysis

### Aggregate Results

| System                 | Mean Overall | Mean Correctness | Mean Completeness | Mean Consistency | Mean Latency ms | Mean Logs | LLM Step Proxy | Failure Rate |
|------------------------|--------------|------------------|-------------------|------------------|-----------------|-----------|----------------|--------------|
| Full Multi-Agent       | 0.99         | 0.97             | 1.00              | 1.00             | 10.21           | 13.4      | 2.8            | 0.00         |
| Simplified Multi-Agent | 0.99         | 0.97             | 1.00              | 1.00             | 11.90           | 14.8      | 1.8            | 0.00         |
| Single-Agent           | 0.94         | 0.93             | 1.00              | 0.80             | 9.36            | 7.4       | 1.0            | 0.00         |

### Per-Test Results

| Case | System           | Overall | Correctness | Completeness | Consistency | Logs | QA       | Browser | Topic                     |
|------|------------------|---------|-------------|--------------|-------------|------|----------|---------|---------------------------|
| T1   | Full Multi-Agent | 1.00    | 1.00        | 1.00         | 1           | 15   | approved | success | Pilates                   |
| T1   | Single-Agent     | 1.00    | 1.00        | 1.00         | 1           | 8    | approved | success | Pilates                   |
| T2   | Full Multi-Agent | 1.00    | 1.00        | 1.00         | 1           | 14   | approved | success | silver earrings           |
| T2   | Single-Agent     | 1.00    | 1.00        | 1.00         | 1           | 8    | approved | success | silver earrings           |
| T3   | Full Multi-Agent | 1.00    | 1.00        | 1.00         | 1           | 14   | approved | success | doğa sporları ekipmanları |
| T3   | Single-Agent     | 1.00    | 1.00        | 1.00         | 1           | 8    | approved | success | doğa sporları ekipmanları |
| T4   | Full Multi-Agent | 0.94    | 0.83        | 1.00         | 1           | 13   | approved | success | a reformer bed            |
| T4   | Single-Agent     | 0.68    | 0.67        | 1.00         | 0           | 8    | approved | success | your update               |
| T5   | Full Multi-Agent | 1.00    | 1.00        | 1.00         | 1           | 11   | approved | skipped | jewelry                   |
| T5   | Single-Agent     | 1.00    | 1.00        | 1.00         | 1           | 5    | approved | skipped | jewelry                   |

### When the Multi-Agent System Performed Better

The multi-agent system performed better on mixed-intent prompts. The clearest example is T4. The user supplied an exact caption, "Join my class!"; and separately requested a reformer bed image. The multi-agent Task Manager preserved the exact caption while still extracting the image topic as a reformer bed. The single-agent system preserved the caption but lost the visual topic and used your update, which reduced its consistency score.

This shows the value of task decomposition. The Task Manager separates caption handling from media handling. The Writer can be skipped, while the Visual Prompt Engineer and Media Creator still run.

### When the Single-Agent System Performed Better

The single-agent system was simpler and cheaper for straightforward cases. It averaged 7.4 logs and an LLM step proxy of 1.0. The full multi-agent system averaged 13.4 logs and an LLM step proxy of 2.8. In T1, T2, T3, and T5, the single-agent system produced complete and correct outputs.

This means the single-agent approach is a strong baseline when the prompt is simple, explicit, or does not require multiple independent checks.

### Coordination Issues

The multi-agent system has more coordination overhead. It must validate Researcher, Writer, Visual Prompt Engineer, Media Creator, QA, and Browser outputs. More steps create more places where latency, retries, or handoff mistakes can occur. However, the same checkpoints also improve observability. If QA rejects the caption, the Task Manager can send only the caption-related output back through repair. If media is missing, only the Media Creator needs to be retried. In the single-agent system, internal mistakes are harder to isolate because all responsibilities are combined.

### Hallucination and Inconsistency Patterns

The main inconsistency appeared in the single-agent T4 result. The single agent focused on preserving the exact text but failed to keep the separate image requirement as the topic. This is a common risk when a single model must handle multiple instructions at once.

During later manual testing, another important issue appeared in the multi-agent visual prompt stage: "outdoor Pilates" was initially converted into an indoor Pilates studio because the visual prompt rule prioritized the word "Pilates" before the word

"outdoor." This was fixed by enforcing a design principle: explicit user constraints such as outdoor/indoor environment must override generic topic defaults.

Qualitative Visual Comparison

I also reviewed manual UI screenshots from one single-agent run and one multi-agent run. I did not include these screenshots in the quantitative score because the experiment metrics do not measure image aesthetics directly. However, they are useful as qualitative evidence.

| Observation Area      | Single-Agent Screenshot                                                                                | Multi-Agent Screenshot                                                                                  | Interpretation                                                                                                                                       |
|-----------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prompt/topic handling | Topic appeared as this and the caption became generic: "Bring more intention into your day with this." | Topic also appeared as this and the caption was similarly generic.                                      | Both systems can fail when the prompt is parsed too broadly or when the input lacks a clear extracted topic.                                         |
| Image relevance       | Generated a realistic person sitting on a sofa.                                                        | Generated a generic desk/workspace image.                                                               | The single-agent image was more human/lifestyle-oriented, but neither image should be treated as a strong match without the original prompt context. |
| Workflow trace        | The UI clearly showed a single unified agent.                                                          | The UI showed handoffs across several agents, including skipped steps.                                  | The screenshots are useful for explaining architecture difference, not for proving visual quality.                                                   |
| Failure insight       | One agent produced a complete output, but the topic/caption were too vague.                            | Multiple agents completed the pipeline, but the final media still depended on correct topic extraction. | Multi-agent design improves observability, but it does not automatically guarantee better images if the initial topic is wrong.                      |

For presentation, these screenshots should be shown as a **manual failure/diagnostic case**, not as the main experimental result. The correct claim is: "The controlled experiment measures topic/caption/media existence and consistency. The screenshots show that future work should add a human visual-quality rubric."

Trade-Offs

The results show a clear trade-off:

- The single-agent system is faster, cheaper, and easier to implement.
- The multi-agent system is more robust for mixed requirements and easier to debug.
- The multi-agent system costs more in coordination, logs, and model-like steps.
- Dynamic routing is necessary; a multi-agent system should not blindly run every agent.

Part 5 - Ablation / Simplification Study

For the simplification study, I modified the multi-agent system by removing two components:

- Trend Researcher Agent
- Visual Prompt Engineer Agent

The simplified multi-agent system still used:

- Task Manager
- Writer
- Media Creator
- QA Agent
- Browser Operator

Ablation Results

| System                 | Mean Overall | Mean Logs | LLM Step Proxy | Failure Rate |
|------------------------|--------------|-----------|----------------|--------------|
| Full Multi-Agent       | 0.99         | 13.4      | 2.8            | 0.00         |
| Simplified Multi-Agent | 0.99         | 14.8      | 1.8            | 0.00         |

In this deterministic test set, the simplified multi-agent system matched the full system’s average overall score while reducing the LLM step proxy from 2.8 to 1.8. This suggests that Trend Researcher and Visual Prompt Engineer are not always necessary for simple prompts.

However, this does not mean these agents are useless. The Trend Researcher is useful when the user asks for current trends, hashtags, or popular topics. The Visual Prompt Engineer is useful when real image generation quality matters, because text-to-image models are sensitive to camera, lighting, composition, and negative prompt details.

Necessary Components

The experiment suggests the following components are core:

| Component              | Necessity                                                     |
|------------------------|---------------------------------------------------------------|
| Task Manager           | Necessary for routing, validation, skipping steps, and repair |
| Writer                 | Necessary when the caption is missing                         |
| Media Creator          | Necessary when media is requested                             |
| QA Agent               | Necessary for safety and quality validation                   |
| Browser Operator       | Necessary only when upload/publish is requested               |
| Researcher             | Conditional; useful for trend-based prompts                   |
| Visual Prompt Engineer | Conditional; useful for high-quality real image generation    |

The most important design lesson is that agents should be conditional. The system should start with the minimum required steps and add specialized agents only when they improve the output.

## Supplementary Real API Trial

I also ran the experiment with real external APIs. This run used NVIDIA/Mistral provider paths and is useful for observing operational behavior, but it is not the primary baseline because external APIs can fail or slow down independently of the architecture.

Output directory:

```
projects/Smart_Social_Media_Agent_Orchestrator/outputs/assignment3_experiment/20260514_151828
```

| System                 | Mean Overall | Mean Correctness | Mean Completeness | Mean Consistency | Mean Latency ms | Failure Rate |
|------------------------|--------------|------------------|-------------------|------------------|-----------------|--------------|
| Full Multi-Agent       | 0.99         | 0.97             | 1.00              | 1.00             | 89165.11        | 0.00         |
| Simplified Multi-Agent | 0.99         | 0.97             | 1.00              | 1.00             | 36903.27        | 0.00         |
| Single-Agent           | 0.64         | 0.73             | 0.55              | 0.80             | 12981.24        | 0.60         |

The single-agent score dropped because three image-generation runs failed with a Mistral connection/DNS error: nodename nor servname provided, or not known. This is an external service/network failure, not necessarily a pure architecture failure. The real API trial demonstrates that production systems need retries, fallback providers, and error isolation.

## Part 6 - Reflection: When Do Agentic Systems Make Sense?

Multi-agent systems are beneficial when a task contains multiple distinct responsibilities that can be checked independently. In this project, planning, research, writing, visual prompt engineering, media generation, QA, and browser operation are different types of work. Splitting them into agents makes the system easier to inspect, repair, and extend.

Multi-agent systems introduce unnecessary complexity when the task is simple or already well specified. If the user only needs a short caption, a single agent is enough. Running multiple agents in that case wastes time, tokens, and engineering effort.

My design recommendations for future agentic systems are:

- Use a Task Manager only when routing decisions matter.
- Give each agent a narrow responsibility and structured output.
- Validate after every step.
- Add repair paths for common failures.
- Skip agents dynamically when their work is not needed.
- Treat external API failures separately from model reasoning failures.
- Prefer the simplest architecture that meets the quality and reliability requirement.

For this social media automation task, the multi-agent system is justified for vague, mixed, or incomplete prompts. The single-agent system remains valuable as a baseline because it is simpler and cheaper for straightforward prompts. Therefore, the best design is not “more agents everywhere,” but **dynamic orchestration with the smallest sufficient number of agents**.

## Responsible Use of AI

AI tools were used to help implement and test the systems, but the evaluation design, metrics, interpretation, and final conclusions are my responsibility. I can explain each metric, why deterministic testing was used as the baseline, why real API results were treated separately, and why the ablation study shows that some agents are conditional rather than always necessary.